

Laboratoire 2 : gestionnaire de tickets

6GEI311 Architecture des logiciels, UQAC, automne 2026. SAWADOGO Christian Kevin et SOPPEWE TCHAWOU JEREMIE ARNO

<https://github.com/Mrpropre12/lab2-architecture-de-logiciel.git><https://github.com/Mrpropre12/lab2-architecture-de-logiciel.git>

I. Critique du diagramme de classes initial

Le diagramme initial (User, Admin, Ticket) suffit pour coder la partie 1, mais il ne garantit ni le cycle de vie décrit dans l'énoncé, ni la description riche du ticket, ni le contrôle de qui fait quoi. Les décisions reposent sur des chaînes de caractères : chaque nouveau rôle, statut ou règle oblige à modifier plusieurs classes, ce qui va à l'inverse de la modifiabilité visée par le laboratoire.

Constat dans le diagramme	Conséquence
role : String dans User, et une classe Admin séparée qui recopie name et email avec son propre adminID, sans lien avec User.	Deux façons concurrentes de représenter un rôle. Un admin ne peut pas créer de ticket. Le rôle Développeur de l'énoncé (assigner, valider, fermer) n'existe pas : ces actions sont données à Admin seul.
status : String et updateStatus(status) public.	Toute valeur et toute transition sont acceptées : on passe d'OUVERT à TERMINÉ sans VALIDATION, et « ASSIGNE » au lieu de « ASSIGNÉ » crée un statut fantôme. Le cycle de vie n'est écrit nulle part.
Les méthodes du ticket ne reçoivent pas l'acteur (assignTo, updateStatus, addComment).	Le ticket ne sait pas qui agit, il ne peut donc rien refuser.
description : String.	Impossible de joindre du texte, des captures ou des vidéos, ni d'exprimer leurs chargements différents. Rien pour consulter ou exporter en PDF.
Associations : views de multiplicité 1 côté User, composition (losange plein) entre Admin et Ticket, aucun lien vers l'auteur ni vers l'assigné.	Un ticket ne serait vu que par une personne. Supprimer l'admin détruirait ses tickets. assignTo(user) reçoit un utilisateur qu'il ne conserve pas.
addComment(comment : String) sans classe Comment.	Le commentaire n'est pas gardé : ni auteur, ni date, rien à afficher.
Tous les attributs sont publics (+), y compris status, ticketID et les dates.	N'importe quel code modifie l'état sans passer par les règles.
createTicket(ticket : Ticket) reçoit un ticket déjà construit ; viewTicket et updateTicket renvoient void ; viewAllTickets() est dans Admin.	Aucun registre ne génère les identifiants ni ne retrouve un ticket. updateTicket ne dit pas ce qu'il modifie.
priority : String.	Valeurs libres, comme pour le statut.

II. Améliorations du diagramme et justifications

Le diagramme final compte 30 classes réparties en trois paquetages. Chaque changement ci-dessous répond à un constat de la section I ; la dernière colonne dit quelle bonne pratique nous avons appliquée et comment nous l'avons instanciée.

Changement	Justification : bonne pratique et instanciation
Admin disparaît. User agrège des objets Role : interface Role, classe abstraite BaseRole, puis ReporterRole, DeveloperRole, AdminRole. Une énumération Permission liste les 16 droits.	RBAC par composition plutôt qu'héritage. Chaque rôle déclare un ensemble figé (FrozenSet) de permissions ; canGlobally(p) interroge les rôles. Ajouter un rôle = ajouter une classe, sans toucher User ni Ticket (ouvert/fermé). Un compte peut cumuler des rôles ; AdminRole implique DeveloperRole.
status : String devient le patron État : TicketState abstrait et cinq états (Ouvert, Assigné, Validation, Terminé, Fermé). L'énumération Status sert à l'affichage.	Le cycle de vie de l'énoncé devient du code. TicketState refuse toute transition par défaut ; chaque état redéfinit seulement ce qu'il permet (ValidationState.validate mène à TERMINÉ). updateStatus disparaît : submit, validate, reject, close, reopen délèguent à l'état courant. Les transitions sont de visibilité paquetage (~).
Chaque méthode publique reçoit l'acteur en premier paramètre et renvoie un résultat (boolean, objet [0..1] ou liste).	Médiation complète et moindre privilège. La méthode privée authorize(actor, p) vérifie le droit global ; les règles propres à la personne (auteur, assigné, visibilité) sont dans isAuthor, isAssignee, isVisibleTo.
description : String devient une liste de Content : TextFileContent, ImageFileContent, VideoFileContent, créés par ContentFactory.fromPath et confinés par PathGuard.	Polymorphisme et Fabrique , comme demandé en 2.1. Chaque type garde un chemin et charge différemment : le texte est lu tout de suite, l'image au premier affichage puis mise en cache (unload() libère), la vidéo n'est jamais lue. Les constructeurs sont ~ : la fabrique est la seule voie de création et choisit le type selon l'extension.
Patron Visiteur : TicketVisitor (visitTicket, visitComment, visitText, visitImage, visitVideo), spécialisé en ViewVisitor et ExportVisitor.	Consulter et exporter sans alourdir Ticket ni les contenus. ScreenRenderVisitor produit le texte à l'écran, PDFExportVisitor le PDF. Un export HTML serait un nouveau visiteur, sans rien modifier (ouvert/fermé).
Associations refaites : auteur 1, assigné 0..1, commentaires et contenus en composition 0..* ; plus de composition avec l'admin. Nouvelle classe Comment (auteur, texte, date).	La composition traduit la vraie possession : un commentaire n'existe pas sans son ticket, alors qu'un ticket survit à son assigné. La multiplicité 1 de views disparaît : la visibilité est une règle, pas une association.
Attributs privés (-), accesseurs en lecture seule, mutateurs ~ (setState, setAssignee, touch).	Encapsulation : l'état ne change qu'en passant par les règles.
Deux registres : UserRegistry (inscription, email {unique}, gestion des rôles, désactivation) et TicketRegistry (création, recherches, archivage).	Responsabilité unique et injection de dépendances. Les registres génèrent les identifiants. TicketRegistry(users) reçoit le registre des comptes et refuse tout compte qui n'y est pas. Un Singleton a été essayé puis rejeté (état global, une seconde exécution plantait).

Changement	Justification : bonne pratique et instanciation
Archivage au lieu de suppression : permission ARCHIVE (admin), seulement pour un ticket TERMINÉ ou FERMÉ, définitif.	Garder la trace d'un ticket résolu. Un ticket archivé reste consultable et exportable, rien d'autre ; la règle est tenue en un seul point, autorize.
Trois paquetages : « Qui agit », « Le ticket », « Consultation et export ».	Forte cohésion, faible couplage, dépendances acycliques. Le contrat du Visiteur est dans « Le ticket » et les visiteurs concrets en dépendent (inversion des dépendances).

La figure suivante résume l'organisation en paquetages et le sens des dépendances.

Le contrat du Visiteur reste dans « Le ticket » : aucune dépendance ne boucle

Trois paquetages ; une flèche se lit « dépend de »

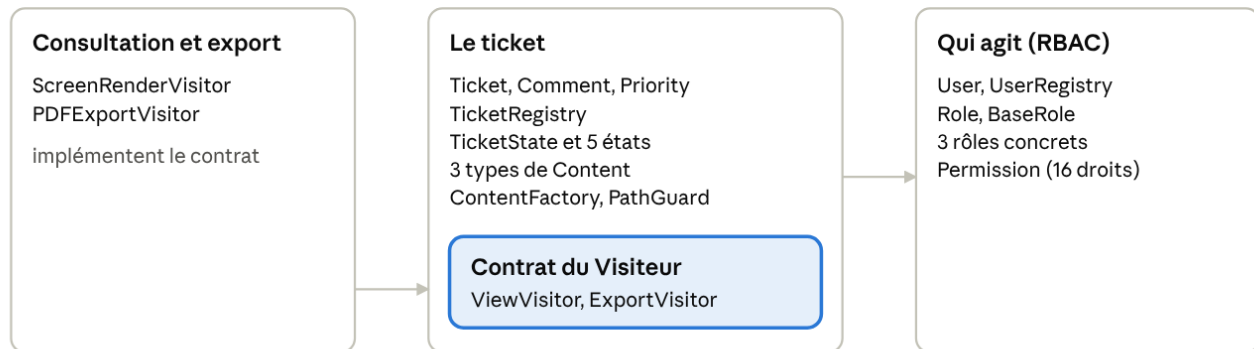


Figure 1. Paquetages du diagramme final et sens des dépendances.

III. Autres améliorations du code et de la conception

Au-delà du diagramme, nous avons fixé des règles communes à tout le code, durci l'accès aux fichiers et construit une console qui sert de démonstration.

- **Un seul ordre de contrôle.** Chaque action vérifie, dans cet ordre : le droit du rôle, les arguments, l'état du ticket, la règle propre à la personne (auteur, assigné, visibilité), puis applique l'effet et met à jour la date. Un refus arrive donc toujours avant toute modification.
- **Une seule convention de refus.** Une action refusée renvoie False, None ou une liste vide, sans effet de bord, et son motif est écrit dans le journal (logging). Aucune exception ne sort du modèle pour une action interdite.
- **Invariants du domaine.** L'assigné doit avoir le droit SUBMIT ; l'assigné ne valide ni ne rejette son propre travail ; la réouverture remet le ticket à OUVERT sans assigné ; les motifs de rejet, fermeture et réouverture deviennent des commentaires ; le dernier administrateur actif ne peut être ni rétrogradé ni désactivé ; l'email est unique.
- **Fichiers joints.** La fabrique vérifie l'extension, la signature réelle du fichier et la taille (texte 1 Mo, image 10 Mo et 25 mégapixels, vidéo 500 Mo). PathGuard refuse tout chemin qui sort du dossier donnees/televersements. Un fichier illisible ne fait pas planter le programme : la fabrique renvoie None.
- **Texte saisi et export.** Les titres, noms et commentaires contenant des caractères de contrôle ou bidirectionnels sont refusés (_texte.py), ce qui empêche de falsifier l'affichage. Chaque PDF reçoit un nom unique dans donnees/exports, l'utilisateur ne choisit donc pas où écrire.
- **Du diagramme au code.** Nous avons fixé une correspondance stricte : +, ~, - deviennent nom, _nom, __nom ; camelCase devient snake_case (PEP 8) ; <<create>>, equals et hash deviennent __init__, __eq__ et __hash__ ; un accesseur devient une propriété en lecture seule. Un paquet Python par paquetage UML.
- **Console par rôle.** main.py lance une console interactive avec cinq comptes et deux tickets de départ. Le menu ne montre que les actions permises par les rôles du compte connecté ; la touche T affiche aussi les autres,

marquées *. Chaque action a son propre écran avec OK, ou REFUSE et le motif donné par le modèle. Le modèle revérifie toujours : l'interface filtre pour le confort, le modèle protège (défense en profondeur).

- **Vérification.** 156 tests unitaires, dont un test qui lit le diagramme PlantUML et vérifie que les 30 classes existent dans le code avec leurs méthodes, et un test qui vérifie que les imports respectent le sens des paquets. Les 108 erreurs introduites volontairement dans le code (tests de mutation) ont toutes été détectées. Trois relectures indépendantes ont trouvé 12 défauts, dont 3 graves (un compte venu d'un autre registre avait tous les droits, un fichier joint pouvait être remplacé après vérification, un fichier illisible faisait planter le programme), tous corrigés et couverts par un test. Les tests ne sont pas dans la remise, qui ne contient que le code utile.

IV. Leçons apprises

1. **Critiquer un diagramme, c'est le confronter à l'énoncé.** Chaque phrase de l'énoncé (qui assigne, qui valide, quels statuts) doit trouver sa place dans le modèle. Le diagramme initial se codait sans peine, mais la moitié des règles n'y figuraient pas.
2. **Une chaîne de caractères cache une décision.** Remplacer status, role et priority par des types (états, rôles, énumérations) rend une transition interdite impossible à écrire, au lieu de devoir la détecter partout.
3. **Un patron se justifie par un changement probable.** État, Fabrique, Visiteur et RBAC répondent chacun à une évolution plausible : un nouveau statut, un nouveau type de fichier, un nouveau format d'export, un nouveau rôle. À l'inverse, le Singleton n'apportait que de l'état global, et nous l'avons retiré au profit de l'injection.
4. **Le code corrige aussi le diagramme.** L'implémentation a fait remonter des oublis (un chemin refusé ne renvoie rien, le registre des comptes doit être injecté). Le test qui compare le code au fichier PlantUML nous a gardés alignés jusqu'à la fin.
5. **Un test qui passe ne prouve pas qu'il teste.** Les tests de mutation nous ont permis de vérifier que chaque règle importante fait échouer au moins un test si on la casse.
6. **L'interface n'est pas une barrière de sécurité.** Masquer un bouton améliore le confort, mais c'est le modèle qui doit refuser. Une relecture par un regard extérieur a trouvé trois défauts graves que nos propres tests ne visaient pas.

Pour le prochain laboratoire, l'interface graphique pourra remplacer la console sans toucher au modèle ; il faudra ajouter une vraie authentification et une méthode qui donne les actions permises pour un ticket.